

Diagramas de Estado, Transiciones, Eventos y Acciones.

1. Diagrama de Estado

Un **diagrama de estado** es una representación gráfica utilizada en informática y en ingeniería de software para mostrar el comportamiento de un sistema a través de sus distintos estados y cómo cambia entre ellos.

Se utiliza principalmente en el modelado de sistemas dinámicos, especialmente en el lenguaje UML (Lenguaje de Modelado Unificado).

Características principales:

- Representa los diferentes estados de un objeto o sistema.
- Muestra cómo se producen los cambios de estado.
- Permite entender el comportamiento del sistema en diferentes situaciones.

2. Estados

Un **estado** es una condición o situación en la que se encuentra un objeto en un momento determinado.

Ejemplo:

Un semáforo puede tener los estados:

- Rojo
- Amarillo
- Verde

3. Transiciones

Una **transición** es el cambio de un estado a otro dentro del sistema.

Se representan con flechas que conectan los estados.

Ejemplo:

Rojo → Verde → Amarillo → Rojo

4. Eventos

Un **evento** es algo que ocurre y provoca una transición entre estados.

Tipos de eventos:

- Evento externo: generado por el usuario o el entorno.
- Evento interno: ocurre dentro del sistema.

Ejemplo:

- Presionar un botón
- Pasar cierto tiempo
- Recibir una señal

5. Acciones

Una **acción** es una actividad que se ejecuta como resultado de un evento o durante una transición.

Ejemplo:

- Encender una luz
- Mostrar un mensaje
- Guardar datos

6. Relación entre los elementos

- Un **estado** representa una condición.
- Un **evento** provoca un cambio.
- Una **transición** es el cambio entre estados.
- Una **acción** es lo que ocurre durante ese cambio.

7. Ejemplo sencillo

Sistema: Máquina expendedora

- **Estado:** Esperando dinero
- **Evento:** Insertar moneda
- **Transición:** Pasa a “Seleccionar producto”
- **Acción:** Mostrar opciones

8. Importancia

Los diagramas de estado son importantes porque:

- Ayudan a entender sistemas complejos.
- Facilitan el diseño de software.
- Permiten detectar errores antes de programar.
- Mejoran la comunicación entre desarrolladores.